

1. Car Class with Object Creation

```
class Car {  
    String brand;  
    String model;  
    int year;  
  
    public static void main(String[] args) {  
        Car car = new Car();  
        car.brand = "Toyota";  
        car.model = "Camry";  
        car.year = 2023;  
  
        System.out.println("Brand: " + car.brand);  
        System.out.println("Model: " + car.model);  
        System.out.println("Year: " + car.year);  
    }  
}
```

2. Method to Add Two Numbers

```
class AddNumbers {  
    static int addNumbers(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int result = addNumbers(10, 20);  
        System.out.println("Sum: " + result);  
    }  
}
```

3. Default and Parameterized Constructor

Default Constructor: No parameters, initializes with default values

Parameterized Constructor: Accepts parameters to initialize with specific values

```
class Book {
```

```
    String title;
```

```
    String author;
```

Default constructor

```
Book() {
```

```
    title = "Unknown";
```

```
    author = "Unknown";
```

```
}
```

Parameterized constructor

```
Book(String title, String author) {
```

```
    this.title = title;
```

```
    this.author = author;
```

```
}
```

```
void display() {
```

```
    System.out.println("Title: " + title + ", Author: " + author);
```

```
}
```

```
public static void main(String[] args) {
```

```
    Book book1 = new Book();
```

```
    Book book2 = new Book("1984", "George Orwell");
```

```
    book1.display();
```

```
    book2.display();
```

```
}
```

```
}
```

5. Access Modifiers Demo

public: Accessible from anywhere

private: Accessible only within the same class

protected: Accessible within package and subclasses

default: Accessible only within the same package

```
*/
```

```
class AccessModifiersDemo {
```

```
    public String publicVar = "Public";
```

```
    private String privateVar = "Private";
```

```
    protected String protectedVar = "Protected";
```

```
    String defaultVar = "Default";
```

```
    private void privateMethod() {
```

```
        System.out.println("Private method called");
```

```
    }
```

```
    public void publicMethod() {
```

```
        System.out.println("Public method called");
```

```
        System.out.println(privateVar);
```

```
        privateMethod();
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        AccessModifiersDemo obj = new AccessModifiersDemo();
```

```
        System.out.println(obj.publicVar);
```

```
        obj.publicMethod();
```

```
    }
```

```
}
```

6. Student Class with 'this' Keyword

```
class Student {
```

```
    String name;
```

```
    int age;
```

Using 'this' to refer to instance variables

```
    Student(String name, int age) {
```

```
        this.name = name; // 'this.name' refers to instance variable
```

```
        this.age = age;
```

```
    }
```

Using 'this' to call another constructor

```
    Student(String name) {
```

```
        this(name, 18);
```

```
    }
```

```
    void display() {
```

```
        System.out.println("Name: " + this.name + ", Age: " + this.age);
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Student s1 = new Student("Alice", 20);
```

```
        Student s2 = new Student("Bob");
```

```
        s1.display();
```

```
        s2.display();
```

```
    }
```

```
}
```

7. Final Keyword Examples

final class: Cannot be inherited

final method: Cannot be overridden

final variable: Cannot be reassigned (constant)

```
final class FinalClass {  
    // Cannot extend this class  
}
```

```
class FinalDemo {  
    final int MAX_VALUE = 100; // Constant variable  
  
    final void finalMethod() {  
        System.out.println("This method cannot be overridden");  
    }  
  
    public static void main(String[] args) {  
        FinalDemo obj = new FinalDemo();  
        System.out.println("Max Value: " + obj.MAX_VALUE);  
        // obj.MAX_VALUE = 200;  
        obj.finalMethod();  
    }  
}
```

8. Factorial using Recursion

Recursion: A method calls itself

Iteration: Uses loops (for, while)

Recursion is more elegant but uses more memory (call stack)

Iteration is more efficient for simple repetitive tasks

```

class Factorial {

    static int factorial(int n) {

        if (n == 0 || n == 1) {

            return 1; // Base case

        }

        return n * factorial(n - 1); // Recursive call

    }

    public static void main(String[] args) {

        int num = 5;

        System.out.println("Factorial of " + num + " = " + factorial(num));

    }

}

```

9. instanceof Operator with Inheritance

```

class Animal {

    void sound() {

        System.out.println("Animal makes sound");

    }

}

```

```

class Dog extends Animal {

```

```

    void sound() {

        System.out.println("Dog barks");

    }

}

```

```

    public static void main(String[] args) {

```

```

        Dog dog = new Dog();

```

```

        System.out.println("dog instanceof Dog: " + (dog instanceof Dog));

```

```

        System.out.println("dog instanceof Animal: " + (dog instanceof Animal));
    }
}

```

```
}  
}
```

10. Complete Class Demonstration

```
class Person {  
    // Variable declaration  
    String name;  
    int age;  
  
    // Constructor  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Method creation  
    void introduce() {  
        System.out.println("Hi, I'm " + name + " and I'm " + age + " years old.");  
    }  
  
    public static void main(String[] args) {  
        // Object creation  
        Person person = new Person("John", 25);  
  
        // Using the method  
        person.introduce();  
    }  
}
```